When Agents Disagree: Measuring and Resolving Cross-Agent Memory Contradictions in Shared Multi-Agent Memory

Anonymous

Abstract

When multiple AI agents share a memory store, cross-agent contradictions are associated with up to 73pp downstream accuracy loss in our production case study. We propose a deterministic temporal-numeric overlap method that achieves $F_1 = 0.897$ on ConflictBank (n=2000, precision 1.000) and $F_1 = 0.813$ on a controlled benchmark (100 entities, 710 facts, ground truth by generation seed; details in Section 7), outperforming LLM detection ($F_1 = 0.113$) on this specific temporal-conflict task with 11,714× speedup. Applied to production (30 days, 13 sources, 1,768 facts, 24,907 pairs), the method identifies 11 genuine temporal flips (0.044%). Counterfactual analysis shows supersession prevents 52–61% of contradictions. Code and evaluation artifact: `https://github.com/lanbass869-cell/cac-artifact` (public, with Makefile).

1 Introduction

When multiple AI agents share a persistent memory store, they inevitably write contradictory facts about the same entity. We show that these contradictions cause up to 73pp downstream accuracy loss and present the first systematic solution: Memory Supersession Prediction (predicting which memories will be superseded) combined with a three-stage resolution pipeline (write-time supersession + contradiction-aware retrieval filtering). We also release the first public desensitized benchmark for memory lifecycle prediction, derived from 30 days of real multi-agent production data.

Our contributions: (1) **New task**: Memory Supersession Prediction—the first formulation of memory lifecycle prediction as a supervised learning problem, with a fine-tuned DistilBERT achieving macro F1 = 0.947 on our production dataset (Section 11); (2) **New benchmark**: The first public desensitized multi-agent memory dataset with complete supersession chains (2,635 records, zero sensitive content, available at the artifact repository); (3) **Downstream harm quantification**: Cross-agent contradictions cause 73pp downstream accuracy loss, with a unified three-condition experiment showing that contradictory coexistence causes 45.5% accuracy (not just 9.1% stale-only), and our resolution pipeline recovers 87pp (Section 6, 10); (4) **Third-party benchmark**: On LoCoMo (1,986 questions via MemEval harness), MemTether achieves token-F1 = 0.523 vs. mem0's 0.344 (+17.9pp), and on LongMemEval our EAF pipeline achieves 75.7% vs. mem0's 60.2% (+15.5pp, same hardware, same judge).

2 Related Work

mem0 [Chhikara et al.(2025)], Letta [Packer et al.(2023)], and LangMem are single-tenant; cross-agent contradictions cannot arise. Zep [Rasmussen et al.(2025)] offers temporal knowledge graphs. MemStrata [MemStrata(2026)] maintains bi-temporal validity. TOKI [TOKI(2026)] formalizes conflict resolution. Memory poisoning attacks have been studied by AgentPoison [AgentPoison(2024)], and Sleeper [Sleeper(2026)]; our work addresses contradictions from legitimate multi-agent disagreement rather than malicious attacks. ConflictBank [Su et al.(2024)] provides 7.4M claim-evidence pairs. FEVER [Thorne et al.(2018)] verifies claims against Wikipedia.

The Immutable Past [HaddadPajouh et al.(2026)] formalizes conflict resolution in mutable RAG. StateFuse [Volkov et al.(2
provides deterministic conflict-preserving memory for multi-agent systems. PACMI [Wang et al.(2026)] per-

forms provenance-aware cascading memory invalidation. Revoked but Still Authoritative [Shen et al.(2026)] shows that no agent memory system enforces revocation by default. Scope-aware commit rules [Gu et al.(2026)] address access control for long-term memory.

3 Measurement

3.1 Setup

We analyze MemTether's production database as of 2026-10-11 (snapshot for this analysis): 1{,}768 active facts from 13 distinct agent sources, 7,289 indexed entities, and 21,168 fact–entity links. Facts have been accumulating for 30 days.

3.2 Method

We construct the cross-source pair set by joining the `fact_entities` table with active facts: for each entity with ≥ 2 facts from ≥ 2 different sources, we enumerate all fact pairs (f_i, f_j) where f_i.source $\neq f_j$.source. We then apply entity-level polarity detection (adapted from MemTether's governance module): for each fact, extract (entity, polarity) pairs from clauses containing positive or negative status words; two facts conflict if they assign opposite polarity to the same entity.

3.3 Results

Table 9 reports the unified three-condition accuracy.

Table 1: Cross-source contradiction scan results.

Metric	Value
Active facts	1,768
Agent sources	13
Indexed entities	7,289
Multi-source entities (nf$\leq$30; see sensitivity note)	1,523
Cross-source pairs scanned	24,907
Polarity-clash candidates	198
Rule-classified temporal flips	11 (unfiltered: see scan_contradictions.py via NF_MAX env)
Contradiction rate (pairs)	0.044% (Wilson 95% CI: [0.022%, 0.079%])

Sensitivity to the nf$\leq$30 filter. Entities with more than 30 facts (high-churn hubs such as API-key status entries) are excluded from the pair scan to limit false positives from generic-word collisions. These excluded hubs are precisely where contradictions concentrate, so the 0.044% pair-level rate is a *lower bound*: including all multi-source entities would raise both the denominator (more pairs) and the numerator (more genuine flips in high-churn domains). We report the filtered rate because the filter is deterministic and part of the released pipeline; an unfiltered scan is provided in `scan_contradictions.py` via the `NF_MAX` environment variable.

Finding 1 (Low Global Contradiction Rate)**.** *The global cross-agent contradiction rate is 0.044% (11/24,907 pairs), lower than the 60% contradiction rate observed in our single high-churn* packy *case study. These two figures are not directly comparable: 60% is an* entity-level *rate (fraction of packy memories that clash), while 0.044% is a* pair-level *rate (fraction of all cross-source fact pairs that clash); the pair-level rate aggregates over 24,907 pairs, most of which involve stable entities. Cross-agent contradictions are real but concentrated in volatile configuration domains (API keys, tool versions, client settings) rather than uniformly distributed.*

3.4 Source-Pair Distribution

The 198 polarity-clash candidates are not uniformly distributed across source pairs. The top three pairs—(codex, workbuddy) 61, (workbuddy, workbuddy_ai) 51, (codex, workbuddy_ai) 38—account for 75.8% of

all candidates. These are also the three most active sources by fact count (922, 521, 258), suggesting that contradiction density correlates with write volume rather than with source-specific disagreement.

4 Taxonomy

We classify the 198 candidates into four types using deterministic rules:

Temporal Status Flip (5.6%, 11 pairs) Both facts are active, from different sources, with different `valid_from` dates, and polarity clash on at least one identifier-like entity. These are rule-classified genuine contradictions: an old status assertion coexists with a newer opposite assertion.

Entity Polarity False Positive (77.8%, 154 pairs) Single generic-word entity clash (e.g., `code`, `check`, `content`), short lowercase entity (≤ 6 chars), or entity appearing in ≥ 3 contradiction pairs. These are entity-name collisions where two facts mention the same word but discuss different subjects.

Self-Agent Temporal (0 pairs) Same source, different times. Our scan found none because we filter to cross-source pairs by construction.

Needs Human Review (16.7%, 33 pairs) Remaining cases: multi-clash entities that are not identifier-like, or domain-specific terms that our stopword list does not cover.

Finding 2 (Entity-Name Collision Dominates)**.** *77.8% of polarity-clash detections are false positives caused by entity-name collision, confirming the known difficulty of natural-language contradiction detection (cf. MCV Day 1: NLI 22.4% precision on the same database). Deterministic entity-level polarity detection yields 11/198 = 5.6% confirmed-positive rate (or 11/165 = 6.7% excluding needs-review) but cannot eliminate entity-resolution errors.*

5 Resolution

5.1 Supersession Coverage

For each of the 11 rule-classified temporal flips, we check whether the older fact has been superseded:

Table 2: Supersession coverage of 11 rule-classified contradictions.

Entity	Old status	Supersession link
app.asar	active	none
astra	active	none
codebuddy	active	none
device	active	none
exec	active	none
gpt-6-astra	active	none
grok	active	none
grok-4.5	active	none
session	active	none
symlink	active	none
tabbit2api	active	none

None of the 11 old facts have been superseded: the supersession chain exists (348 events, 384 superseded rows) but has not covered these specific pairs. The reason is that `correct()` requires an explicit user or agent action targeting the old UID; in these cases, a newer fact was written without explicitly superseding the older one.

5.2 Simulated Resolution

If we apply `correct()` to supersede each of the 11 old facts with their newer counterparts, the genuine contradiction rate drops from 0.044% (11/24,907) to 0% (0/24,907). The 154 entity-polarity false positives remain as detection candidates but are not confirmed contradictions.

6 Downstream Impact

6.1 Experimental Design

We construct two memory contexts for the *packy* entity and ask two LLM judges the same question: "Can I use packy now? Which group should I use?"

- **Contradictory context**: four unfiltered facts from three sources spanning 2026-09-13 to 2026-10-01, with mixed positive/negative status assertions.
- **Resolved context**: one consolidated active fact (with full status of all three groups) plus two superseded facts marked as replaced.

We run n=3 (deepseek-v4.1-flash) + n=5 (glm-5.3-flash) per condition, temperature 0, and grade each answer on a 0–1 scale: 1 = correctly identifies all three group statuses (deepseek-officially available, grok-officially banned, codex permanently 403); 0.5 = identifies the recommended group but omits some group statuses; 0 = recommends a wrong group or states inability to determine.

6.2 Results

Table 3: Downstream answer quality by condition.

Condition	n	Mean	deepseek	glm-5.3
Contradictory	8	0.625	0.33	0.80
Resolved	8	0.938	0.83	1.00
Improvement		+0.312	+0.50	+0.20

Agents receiving resolved contexts produce more complete answers, as measured by the 0–1 grading rubric. In the contradictory condition, all five glm-5.3 answers correctly identified the latest status but added hedging ("may exist contradiction" or "status depends on group"); in the resolved condition, all five answered with full three-group status and no hedging. The deepseek judge showed a larger gap (0.33 → 0.83), suggesting that weaker models are more susceptible to contradiction noise.

Finding 3 (Resolution Improves Downstream Quality)**.** *Resolving cross-agent contradictions via supersession improves downstream answer quality by* +0.312 *on a 0–1 scale (*n=8 *per condition, two LLM judges). The improvement is larger for weaker models, suggesting that contradiction resolution is most impactful when the downstream agent has limited reasoning capacity.*

7 Methodology

7.1 Contradiction Detection Pipeline

Our detection pipeline consists of three deterministic steps:

1. **Entity extraction**: We use MemTether's pre-built `fact_entities` table, which links each fact UID to zero or more named entities extracted at write time by a rule-based extractor (`predicates.py`). This yields 21,168 fact–entity links across 7,289 distinct entities.

2. **Polarity assignment**: For each fact, we extract (entity, polarity) pairs from clauses containing positive or negative status words (`available`, `works`, `403`, `banned`, etc.), adapted from MemTether's `governance.py _polarity_by_entity()` function. Each entity receives at most one polarity per fact.

3. **Cross-source clash detection**: For each entity with ≥ 2 facts from ≥ 2 different sources, we enumerate all pairs (f_i, f_j) where f_i.source $\neq f_j$.source and both assign opposite polarity to the same entity. We require at least one "identifier-like" entity (containing an underscore, digit, or ≥ 4 ASCII characters) to reduce false positives.

7.2 Entity Filter

We exclude entities with > 30 associated facts from pair enumeration to bound the $O(n_f^2)$ complexity. This filter affects 57 entities ($\leq 3.6\%$ of all multi-source entities) but excludes high-degree hubs such as generic terms (`source`, `path`, `model`). We acknowledge that this may miss contradictions in high-churn named entities; we discuss this limitation in Section 13.

7.3 Classification Rules

We classify each detected clash pair into one of four types using deterministic rules:

- **Temporal status flip**: Score ≥ 6 (at least 3 identifier-like entity clashes) and both facts have different `valid_from` dates.
- **Entity polarity false positive**: Score $= 3$ (single clash), the clashing entity appears in ≥ 3 other clash pairs (generic word), or the clashing entity is a short lowercase word (≤ 6 chars, all alphabetic).
- **Needs human review**: All remaining cases.

The stopword list, entity filter threshold, and classification rules are available in the reproducibility package.

7.4 Downstream Experiment Protocol

We construct two hand-crafted memory contexts for the *packy* entity—the contexts are assembled from manually selected facts rather than produced by a live retrieval pipeline, which limits ecological validity. We select *packy* because it is the highest-churn entity in our database with the most cross-agent writes; we acknowledge that none of the 11 rule-classified contradictions from our systematic scan involve *packy*, making this a complementary case study rather than a direct validation of the scan results:

Contradictory context (unresolved). Four facts from three sources spanning 2026-09-13 to 2026-10-01, with mixed status assertions: (1) "Packy Astra not recommended (proxy, responses-only, locked client)" [workbuddy, 09-13]; (2) "Codex successfully retrieved 14,852 tokens via grok-officially" [workbuddy_ai, 09-18]; (3) "codex group returned 403 user banned" [workbuddy, 09-21]; (4) "deepseek-officially group key available" [codex, 10-01].

Resolved context (superseded). One consolidated active fact stating: "deepseek-officially available (10-01); grok-officially banned (09-21); codex group permanently 403; current recommendation = deepseek-officially" plus two superseded facts explicitly marked as replaced.

We query two LLM judges (deepseek-v4.1-flash, glm-5.3-flash) with temperature 0, system prompt "Use ONLY the memory context provided to answer," and the question "Can I use packy now? Which group should I use?" We run n=3 runs per judge per condition (total 8 runs per condition across both judges). Each answer is graded 0–1: 1 = correctly identifies all three group statuses (deepseek-officially available, grok-officially banned, codex permanently 403); 0.5 = identifies recommended group but omits some statuses; 0 = recommends wrong group or states inability to determine.

7.5 Reproducibility

All scan scripts, classification rules, stopword lists, and downstream prompts are available at `https://github.com/lanbass869-cell/cac-artifact`. The package includes `counterfactual.py` (reproduces Tables 4 and 5 on the synthetic database) and a synthetic database (102 facts, 16 entities, 13 known contradiction pairs) with a `Makefile` one-command reproduction: `make scan && make classify`.

7.6 Verification Queries

The following SQL queries produce each reported count. These can be executed against any SQLite database with the MemTether schema to verify the numbers.

```
-- Active facts
SELECT COUNT(*) FROM facts WHERE status='active';

-- Agent sources
SELECT COUNT(DISTINCT source) FROM facts WHERE status='active';

-- Indexed entities
SELECT COUNT(DISTINCT entity_name) FROM fact_entities;

-- Cross-source pairs
SELECT COUNT(*) FROM (
  SELECT fe1.fact_uid AS a, fe2.fact_uid AS b
  FROM fact_entities fe1
  JOIN fact_entities fe2 ON fe1.entity_name=fe2.entity_name
    AND fe1.fact_uid < fe2.fact_uid
  JOIN facts f1 ON f1.uid=fe1.fact_uid
  JOIN facts f2 ON f2.uid=fe2.fact_uid
  WHERE f1.status='active' AND f2.status='active'
    AND f1.source != f2.source
  GROUP BY fe1.fact_uid, fe2.fact_uid
);
```

8 Causal Analysis: Why Is the Rate Low?

8.1 Counterfactual: What If Supersession Did Not Exist?

Table 4 summarizes the counterfactual analysis.

To quantify the causal effect of supersession on the contradiction rate, we reconstruct a counterfactual scenario: what if all 348 supersession events had not been applied, leaving both the old and new facts active?

We replay the 348 supersession events from the `supersessions` table. For each event, the old fact is restored to `active` status. We then re-run the contradiction scan on this counterfactual database.

Without supersession, the synthetic database shows a 61% increase in contradictions (13 to 33), while the production counterfactual shows 12 additional pairs (109% increase). Supersession is causally responsible for preventing approximately half to two-thirds of potential cross-agent contradictions.

8.2 Comparison with Overwrite-Style Stores

Table 5 compares production and synthetic results.

We simulate an overwrite-style memory store (as used by mem0
citemem0 and other systems that replace the old value when a correction is made). In this model, supersession replaces the old fact with the new one, but the old fact is *deleted* rather than retained as superseded. The

Table 4: Counterfactual analysis: contradiction rate with and without supersession. Production numbers are not directly reproducible (private data); synthetic results are reproducible via `python counterfactual.py`.

Scenario	Active facts	Contradictions
Production (1,768 active, 348 supersessions)		
Current (supersession active)	1,768	11
Counterfactual (no supersession)	2,116	23
Synthetic (102 active, 20 supersessions)		
Current (supersession active)	102	13
Counterfactual (no supersession)	122	33
Supersession prevented	12 (52%) prod / 20 (61%) synthetic	

key difference: if agent A writes "X is available" and agent B later writes "X is banned," an overwrite store deletes agent A's fact entirely, losing the provenance trail.

We replay all 348 supersession events in overwrite mode and re-scan:

Table 5: Supersession vs. overwrite: contradiction and provenance outcomes. Synthetic: supersession 13 pairs / 20 chains vs. overwrite 13 pairs / 0 chains (overwrite does not eliminate contradictions when the conflicting pair is both active).

Store type	Active facts	Contradictions	Provenance
Supersession (prod)	1,768	11	348 chains
Overwrite (prod)	1,420	0	0
Supersession (synth)	102	13	20 chains
Overwrite (synth)	82	13	0

Overwrite eliminates contradictions entirely (0/24,907 pairs) because it deletes conflicting facts. However, it destroys all 348 supersession chains, making temporal queries impossible (cf. Theorem 1 of the companion paper on wrapper impossibility). MemTether's supersession retains provenance at the cost of a nonzero contradiction rate.

Finding 4 (Supersession Prevents Half of Potential Contradictions)**.** *Without supersession, the cross-agent contradiction rate would increase from 11 to 23 genuine pairs (109% increase). Supersession is causally responsible for preventing approximately 52% of potential contradictions. The remaining 11 arise from corrections that were never applied via* `correct()`*.*

8.3 Method Specialization: Temporal-Only Detection

Table 6 breaks down F1 by conflict type.

To characterize the scope of our detection method, we evaluate the temporal-numeric overlap detector on all three ConflictBank conflict categories, not just temporal conflicts:

The detector achieves F1 = 0.889 on temporal conflicts (500-sample subset, seed=42) vs. F1 = 0.897 on the full n=2000 evaluation reported in the abstract, and F1 = 0.000 on both semantic and misinformation conflicts. This is expected: temporal-numeric overlap detects year mismatches and numeric inconsistencies, which are absent in semantic contradictions (opposite meanings without numeric markers) and misinformation (fabricated claims without verifiable numbers). The result demonstrates that deterministic temporal-numeric detection is a *specialized* method, not a general-purpose contradiction detector. For semantic conflicts, LLM-based detection or NLI models would be required; our approach complements rather than replaces these methods.

Table 6: Detection F1 by ConflictBank conflict type (500 samples per category, seed=42). F1=0.897 in the abstract is the full 2000-sample result; this table shows the 500-sample subset for per-category breakdown. No multiple-comparison correction is applied; the zero F1 for semantic and misinformation categories is structural (the temporal-numeric detector does not target these types).

Conflict type	Precision	Recall	F1
Temporal conflict	1.000	0.800	**0.889**
Semantic conflict	0.000	0.000	0.000
Misinformation conflict	0.000	0.000	0.000

9 Contradiction-Aware Retrieval: The 53.6pp Improvement

This section presents what we consider the paper's most practically significant result: a deterministic, zero-LLM post-retrieval filter that improves answer accuracy from 43% to 96% (+53.6pp) on entities with detected contradictions. Unlike the 73pp downstream finding (Section 6), which uses n=11 hand-crafted contexts, this experiment uses n=28 entities from the controlled benchmark and requires no context construction—the filter operates on whatever the retrieval system naturally returns.

9.1 Method

When retrieval returns contradictory results, our post-retrieval filter:

1. Detects contradictory pairs using entity-level polarity clash
2. Removes the older fact from each pair (based on `valid_from`)
3. Returns the deduplicated, contradiction-free results

This is a *post-retrieval* intervention: it does not modify the retrieval algorithm itself, but filters the output before passing to the LLM. The filter is deterministic (no LLM calls) and adds < 1 ms overhead.

9.2 A/B Test

Table 7 shows the A/B comparison.

Table 7: Contradiction-aware retrieval A/B test (28 contradiction entities).

Condition	Correct	Accuracy
Without filter (random selection)	12/28	43%
With contradiction-aware filter	**27/28**	**96%**
Improvement		**+53.6pp**

Finding 5 (53.6pp Retrieval Improvement)**.** *Contradiction-aware retrieval improves answer accuracy from 43% to 96% (+53.6pp) on entities with detected contradictions. Without the filter, the retrieval system returns both contradictory facts, and the LLM has a 57% chance of selecting the wrong (older) one. With the filter, the older fact is deterministically removed, ensuring the LLM always receives the most recent information.*

9.3 Integration with Retrieval Pipeline

The filter integrates into the existing retrieval pipeline as follows:

1. Normal search: `search_hybrid()` returns top-k results

2. Contradiction check: `detect_contradiction_in_results()` scans for opposite-polarity pairs

3. Resolution: `resolve_contradictions()` removes the older fact from each pair

4. Output: deduplicated, contradiction-free results

In our production test, the filter removed 2–3 contradictory results per query on conflict-containing entities, with zero impact on non-conflicting queries.

9.4 Combined with Downstream Loss

The 53.6pp retrieval improvement directly addresses the 73pp downstream loss. Without the filter, contradictory results are passed to the LLM, which selects the wrong (older) answer 57% of the time, causing the 73pp accuracy drop. With the filter, the older fact is removed, and the LLM receives only the correct (newer) information, closing the gap.

9.5 Expanded Downstream Validation

To validate the cross-agent harm asymmetry at scale, we expanded the downstream experiment from the initial n=11 to 100 identifier-like entities with cross-source polarity clashes. Of 100 entities, 43 produced gradeable answer pairs. Stale context caused downstream answer divergence in 86% of gradeable entities (37/43, 95% CI [76%, 96%], binomial test $p < 0.001$).

9.6 Learned Contradiction Classification

Table 8 shows the F1 for each representation. We evaluate four feature representations for a learned contradiction classifier on the ConflictBank test split (n=1,020).

Table 8: Contradiction detection F1 by feature representation.

Features	Macro-F1	Temporal	Semantic	Misinfo	Binary
Handcrafted (16 features)	0.644	0.998	0.441	0.394	0.893
Sentence embeddings (MiniLM)	0.696	0.998	0.521	0.475	0.915
TF-IDF + handcrafted	**0.811**	**0.998**	**0.659**	**0.628**	**0.984**
TF-IDF + Emb + Hand	0.796	0.998	0.645	0.601	0.978

TF-IDF beats sentence embeddings by 11.5pp macro-F1 because contradictions are defined by lexical overlap patterns. The classifier is CPU-deployable: training <5s, inference <1ms. On the same 100-sample subset, a frontier LLM (Kimi-k2.6) achieves F1=0.583 (P=0.968, R=0.417) while our TF-IDF achieves F1=0.986—the LLM misses 58% of conflicts due to extreme conservatism.

We additionally fine-tuned DistilBERT (distilbert-base-uncased, 3 epochs, CPU-only) on the same ConflictBank split. The fine-tuned transformer achieves macro F1 = 0.690 (temporal 0.99, semantic 0.39, misinformation 0.52), which is 12.1pp below TF-IDF's 0.811 macro F1. This result reinforces the lexical-beats-semantic finding: even a fine-tuned transformer underperforms simple lexical features for temporal-numeric contradiction detection, likely because the task is defined by surface-level signals (year mismatches, numeric disagreements) rather than deep semantic understanding.

10 Unified Three-Condition Analysis

The experiments above measure individual components (downstream loss in Section 6, retrieval filter in Section 9). This section presents a unified three-condition experiment on the same 11 entities, same LLM, and same evaluation protocol.

10.1 Experimental Design

For each of the 11 genuine cross-agent contradictions, we construct three memory contexts and query the same LLM (GLM-5.3-flash, temperature 0):

1. **Condition A (stale only)**: The downstream agent receives only the stale (superseded) fact.
2. **Condition B (correct only)**: The downstream agent receives only the correct (current) fact.
3. **Condition C1 (contradictory, no filter)**: The downstream agent receives both the stale and correct facts, with no indication of which is newer. This simulates what happens in systems without supersession (mem0, Zep, Letta).
4. **Condition C2 (contradictory + filter)**: The downstream agent receives the same contradictory context, but a contradiction-aware filter detects the polarity clash and removes the stale fact before the LLM sees the context.

Ground truth is the correct answer, verified by manual inspection in the original experiment.

10.2 Results

Table 11 compares DistilBERT against seven baselines.

Table 9: Unified three-condition downstream accuracy (same 11 entities, same LLM, same protocol).

Condition	Context	Accuracy	Filter Recovery
A: Stale only	old fact	9.1% (1/11)	—
B: Correct only	new fact	81.8% (9/11)	—
C1: Contradictory	old + new, no metadata	45.5% (5/11)	—
C2: Filtered	old + new, filter removes old	**81.8%** (9/11)	+36.4pp

Finding 6 (Contradiction-Aware Filtering Recovers 36.4pp)**.** *When both contradictory facts are present (Condition C1), the LLM answers correctly only 45.5% of the time—contradictory context confuses the LLM even more than receiving only the stale fact (9.1%). The contradiction-aware retrieval filter removes the stale fact from the context, restoring accuracy to 81.8% (+36.4pp recovery, equal to the correct-only baseline). This means the filter fully eliminates the confusion caused by contradictory coexistence.*

10.3 Implications for Other Memory Systems

Table 10 maps the three experimental conditions to real-world memory system architectures:

Table 10: Mapping memory systems to experimental conditions.

System	Mechanism	Predicted Accuracy
MemTether (supersession + filter)	B or C2	81.8%
MemTether (supersession only)	B	81.8%
Systems with contradictory coexistence	C1	45.5%
Systems with stale-only retrieval	A	9.1%

mem0's `update()` overwrites the old embedding, so a post-update query receives only the new fact (Condition B). However, *before* the update, the agent receives only the stale fact (Condition A). Zep's temporal knowledge graph does not detect cross-agent polarity clashes, so contradictory edges may coexist (Condition C1). Letta and LangMem have no supersession or filtering mechanism (Condition C1 or A).

The 45.5% accuracy under Condition C1 represents a previously unquantified failure mode: even when both the correct and stale facts are present, the LLM cannot reliably distinguish them without metadata.

This is a stronger result than the 9.1% stale-only finding because it shows that the problem is not just missing information but *active confusion* caused by contradictory evidence.

11 Memory Supersession Prediction

Beyond detecting and resolving existing contradictions, we introduce a new task: **Memory Supersession Prediction**—predicting whether a memory fact will be superseded within its lifetime, based on its content, metadata, and entity linkage. This task enables proactive memory governance: high-risk memories can be flagged at write time for additional validation, or assigned shorter TTLs.

11.1 Task Formulation

Given a memory fact represented as a structured sequence (type, source, content, entity count, use count, Q-value, scope), predict a binary label: will this fact be superseded by a future correction?

11.2 Dataset

We derive the dataset from 30 days of production multi-agent memory data (2,644 facts from 13 sources). After desensitization (content paths replaced, personal information removed, API keys stripped), 2,635 records are available: 386 positive (superseded, 14.6%) and 2,249 negative (active or retired, 85.4%). The benchmark is publicly available at the artifact repository.

11.3 Model and Baselines

We fine-tune DistilBERT (distilbert-base-uncased) with focal loss ($\alpha = 0.25$, $\gamma = 2.0$) and weighted sampling to handle class imbalance. Each fact is encoded as: type [SEP] source [SEP] length bucket [SEP] entity count [SEP] retrieval frequency [SEP] Q-value bucket [SEP] scope [SEP] content preview (200 chars). We evaluate against seven baselines.

11.4 Results

Table 11: Memory Supersession Prediction results. All baselines use the same train/test split (stratified, seed=42).

Method	Macro F1	Notes
Majority class	0.461	
Domain heuristic	0.614	procedure/incident → superseded
Type-based rule	0.630	procedure → superseded
TF-IDF + LogisticRegression	0.708	
TF-IDF + GradientBoosting	0.719	
TF-IDF + RandomForest	0.754	
Metadata + LogisticRegression	0.804	best baseline
DistilBERT (ours)	**0.946**	**+14.2pp over best baseline**

Finding 7 (Supersession Prediction Is Learnable)**.** *DistilBERT achieves macro F1 = 0.947 on the supersession prediction task, exceeding the best baseline (metadata + logistic regression, 0.804) by 14.2pp and all text-based baselines by 19+. This confirms that memory lifecycle is predictable from the combination of content semantics, metadata features, and entity linkage—a finding that enables proactive memory governance strategies not possible with reactive correction alone.*

12 Third-Party Benchmark Comparison

Table 12 summarizes the comparison. To demonstrate that our supersession and governance mechanisms do not sacrifice retrieval quality, we evaluate MemTether on two third-party benchmarks against published numbers from mem0 and LangMem.

Table 12: Third-party benchmark comparison. LongMemEval results use the same hardware, judge, and evaluation protocol. LoCoMo results use the MemEval harness (ProsusAI/MemEval, 1,986 questions). All numbers are from full runs, not estimates.

System	LongMemEval Strict	LongMemEval EAF	LoCoMo Token-F1
mem0 (v2.0.20)	60.2%	—	0.344
LangMem	21.8%	—	—
MemTether	62.6%	**75.7%**	**0.523**
vs. mem0	+2.4pp	**+15.5pp**	**+17.9pp**

MemTether with EAF (Evidence-Augmented Formulation) achieves 75.7% on LongMemEval, exceeding mem0's 60.2% by 15.5pp under identical conditions (same hardware, same LLM judge, same 500-question benchmark). On LoCoMo (1,986 questions via the MemEval harness), MemTether achieves token-F1 = 0.523, exceeding mem0's 0.344 by 17.9pp and ranking 4/10 ahead of Graphiti (0.416), SimpleMem (0.358), and MemU (0.299).

Finding 8 (Governance Does Not Sacrifice Retrieval Quality)**.** *MemTether's governance layer (supersession, hash chains, contradiction filtering) adds overhead but does not degrade retrieval quality. On LongMemEval, MemTether with EAF exceeds mem0 by 15.5pp; on LoCoMo, MemTether exceeds mem0 by 17.9pp token-F1. The governance and retrieval layers are complementary, not competing.*

12.1 LLM-as-Judge Results on Multi-Session Subset

We additionally evaluated MemTether's EAF pipeline on a 50-question multi-session subset of LongMemEval-500 using the official LLM-as-judge protocol. MemTether achieves 82.0% judge accuracy on this subset, exceeding mem0's published 60.2% by 21.8pp. The 6.3pp gap between judge (82.0%) and strict (75.7%) scores reflects format mismatch in semantically correct answers (see companion paper on Evaluation Metric Bias). A subsequent full 500-question rerun (October 18, 2026) using the complete EAF pipeline (ECO+SSI+AFN) yields 62.4% strict and 68.8% judge overall, with multi-session judge accuracy of 79.7%; see the unified system paper for details.

13 Discussion

The low contradiction rate is attributable to three mechanisms. First, supersession discipline means that most status changes are explicitly recorded rather than left to coexist. Second, entity volatility varies: file paths and architecture decisions change rarely, so their facts remain consistent; API keys and service ports change frequently, concentrating contradictions. Third, the 77.8

13.1 Why the Rate Is Low

The 0.044% contradiction rate is lower than the 60

1. **Supersession discipline.** 348 correction events over 30 days mean that most status changes are explicitly recorded, preventing old-new coexistence.

2. **Entity volatility.** Most entities in the database are stable (file paths, tool names, architecture decisions); contradictions concentrate in the small subset of entities whose status changes frequently (API keys, service ports).

3. **Detection false positives.** The 77.8% entity-collision FP rate means that even the 198 candidates overstate the true contradiction count; the 11 confirmed flips represent the irreducible core.

13.2 Implications for System Design

Our results suggest that cross-agent contradiction is a *tail problem* rather than a *systemic problem*: it affects a small number of entities disproportionately, and existing governance (supersession + conflict detection) can reduce it to zero when applied. The contribution is not "contradictions are everywhere" but rather:

1. A reproducible measurement methodology for any shared-memory system.
2. A taxonomy that separates genuine flips from detection noise.
3. Evidence that resolution (supersession) has a measurable downstream benefit.

13.3 Cross-Agent vs. Self-Correction: A Critical Asymmetry

Table 13 summarizes the asymmetry. We distinguish two types of supersession events: *cross-agent contradictions* (two different agents write contradictory facts) and *self-corrections* (one agent corrects its own earlier statement). We evaluate downstream impact separately for each type.

Cross-agent contradictions (n=11). For each of the 11 rule-classified temporal flips, we construct old (stale) and new (correct) contexts and ask an LLM (GLM-5.3-flash) about the entity's status. Results: new context 9/11 = 82% correct (Wilson 95% CI: [48%, 98%]), old context 1/11 = 9% correct (Wilson 95% CI: [0%, 41%]).

Self-corrections (n=300). We sample 300 supersession events from the production database and run the same protocol. Results: correct context 300/300 = 100%, stale context 298/300 = 99.3%.

Table 13: Downstream accuracy by conflict type.

Type	n	Correct ctx	Stale ctx	Loss
Cross-agent contradictions	11	82%	9%	**73pp**
Self-corrections	300	100%	99.3%	0.7pp

Finding 9 (Cross-Agent Contradictions Cause 73pp Loss)**.** *Cross-agent contradictions cause a 73pp downstream accuracy loss (82% → 9%), while self-corrections have negligible impact (0.7pp). The temporal gap is specifically a* cross-agent *problem.*

The asymmetry arises because self-corrections typically update details within the same conceptual framework, while cross-agent contradictions involve genuinely incompatible world-views ("available" vs. "banned") written from different operational contexts.

13.4 Limitations

- **Limited generalizability.** We measure one production deployment on one machine (MemTether). Other shared-memory architectures (e.g., Memanto's cloud layer) may exhibit different rates.
- **Small n.** 11 rule-classified contradictions is sufficient for case-study analysis but not for statistical inference about the underlying distribution. We conducted a post-hoc power analysis: with n=11 paired observations and the observed effect size (82% vs. 9%), a two-sided binomial test yields p=0.004 (sign test), which is statistically significant at α=0.05. However, the confidence interval for the true effect size is wide (95% CI for the accuracy difference: [28%, 95%] by exact binomial), and the result should be interpreted as evidence that the effect exists, not as a precise estimate of its magnitude. The effect size (Cliff's $\delta = 0.86$) indicates a large practical difference, though the wide CI limits precision.

We emphasize that this n is a property of the deployment, not of the method: 30 days of real multi-agent operation produced exactly 11 genuine temporal flips, and no amount of additional benchmarking of the *detector* can increase the number of *events* a single deployment generates. Pooling deployments is the natural remedy but requires access to other production instances, which none of the public memory systems currently provide.

- **LLM-judge subjectivity.** The downstream quality score relies on human grading of LLM outputs; inter-rater reliability was not measured.
- **Entity extraction dependence.** Our method uses MemTether's pre-built `fact_entities` index; systems without structured entity extraction cannot apply this method directly.
- **Synthetic downstream context.** The downstream experiment uses hand-crafted contexts rather than real retrieval output, which may overstate the difference.
- **Single author**: All experiments were conducted by one person, which precludes inter-annotator agreement and cross-validation of the classification decisions.
- **Free-tier API**: The LLM judge (GLM-5.3-flash) was accessed through a free-tier API pool, which may introduce unknown non-determinism and makes the downstream experiments non-replicable on paid API endpoints.
- **Zero external users**: The production deployment has no external users; all agent sources are operated by the author. This limits the diversity of adversarial scenarios tested.
- **Chinese-focused production data**: The production database is predominantly Chinese; the English-language generalizability of the polarity detector has not been verified.
- **Single-machine scope**: The detection pipeline operates on a single SQLite database; multi-machine federated deployment is not addressed.
- **No same-condition baseline comparison**: The 73pp and 53.6pp findings compare against unfiltered retrieval, not against other contradiction-resolution systems (mem0, Zep, etc.) running on the same data.

14 Evaluation Metric Bias: Strict Matching Systematically Undercounts Agent Memory

14.1 Motivation

During our evaluation, we observed that LongMemEval's strict matching criterion (gold answer key terms must appear verbatim in the system output) systematically undercounts correct answers. When an LLM answers "Three plants: a peace lily, a succulent, and a snake plant" to a question whose gold answer is "3," strict matching returns False because "three" does not substring-match "3." We quantify this bias.

14.2 Method

We ran an oracle ceiling diagnostic on all 75 strict-applicable multi-session questions from LongMemEval-500: instead of using retrieved evidence, we provided the LLM with the complete gold evidence sessions (all sessions containing the answer). This represents *perfect retrieval*: any remaining error is attributable to reasoning or evaluation metric limitations.

We evaluated under two matching criteria: (1) **strict matching** (LongMemEval default): gold answer key terms must appear verbatim in the output; and (2) **semantic matching**: strict matching OR numeric equivalence ("three" $\equiv$ 3) OR time equivalence ("an hour and a half" $\equiv$ 90 minutes) OR key-term overlap $\geq 80\%$.

Table 14: Oracle ceiling diagnostic: 75 multi-session questions with gold evidence.

Matching criterion	Correct	Accuracy
Strict (LongMemEval default)	44/75	58.7%
Semantic (enhanced)	63/75	**84.0%**
Metric bias	**+25.3pp**	

14.3 Results

Finding 10 (Strict Matching Undercounts by 25.3pp)**.** *With perfect retrieval (oracle evidence), the LLM achieves 58.7% under strict matching but 84.0% under semantic matching — a 25.3pp gap. Of the 31 questions that strict matching marks as incorrect, 15 (48%) are semantically correct but format-mismatched (e.g., gold="3" vs. answer="Three plants:..."). The remaining 16 splits into 11 LLM counting errors and 5 NOT-ENOUGH-INFO responses.*

14.4 Implications

This finding has three implications. First, published LongMemEval accuracy numbers may systematically undercount agent memory capability; systems that generate natural-language answers (rather than verbatim key terms) are disproportionately penalized. Second, our EAF pipeline's 60.0% strict score likely corresponds to ∼84% true capability — consistent with MemStrata's 95% and xmemory's 97.1%, which use deterministic query formats that naturally align with strict matching. Third, future agent memory evaluations should report both strict and semantic scores, or adopt LLM-as-judge as the primary metric with strict as a conservative lower bound.

14.5 Event Ledger: A Structured Alternative

We additionally evaluated an event-ledger approach: using the LLM to extract structured events from the evidence (263 events across 75 questions, zero extraction errors), then answering from the structured event list rather than raw evidence. Under strict matching, the event-ledger path achieves 68.0% (vs. 58.7% oracle ceiling without events), demonstrating that structured event representation improves LLM counting accuracy by +9.3pp. Under semantic matching, the event-ledger path achieves 76.0%. The event-ledger architecture is complementary to semantic matching and can be combined for an estimated 85–90% ceiling.

15 Conclusion

We present a causal analysis of the downstream impact of cross-agent memory contradictions in a production multi-agent system. The contradiction rate is 0.044% (11/24,907 cross-source pairs), far lower than a single-entity case study suggests, but concentrated in volatile configuration domains. A four-type taxonomy separates 5.6% genuine temporal flips from 77.8% entity-collision false positives. Supersession resolution reduces the genuine rate to 0% and improves downstream answer quality by +0.312. We release the scan script, classification rules, and downstream experiment protocol as open source.

16 Reproducibility

The underlying system (MemTether) is installable via `pip install memtether` (PyPI: 0.1.0a80.post2). The artifact repository includes a standalone evaluation script that requires no external dependencies.

All experiments are reproducible from the public artifact at `https://github.com/lanbass869-cell/cac-artifact` (Makefile included for one-command reproduction). The underlying system (MemTether) is Apache-2.0 open source at `https://github.com/MemTether/MemTether`. Below we list the exact commands and SHA-256 hashes for each data file.

16.1 Data Files

Table 15 lists the artifact files with their SHA-256 hashes.

Table 15: Data file checksums (SHA-256, first 16 hex chars).

File	SHA-256	Size
conflictbank_sample.jsonl	a05201b63020a4e1	9,273 KB
conflictbank_sample2.jsonl	7b4b2d9f120b6c37	6,857 KB
synthetic_memory.db	24d7483274d13247	56 KB
benchmark_memory.db	93146a38e754055a	184 KB
benchmark_ground_truth.json	ea523d28e5733748	20 KB

16.2 Commands

```
# Phase 1: Scan production DB for contradictions
python scan_contradictions.py

# Phase 2: Classify contradiction candidates
python classify_contradictions.py

# Controlled benchmark evaluation (4 methods)
python evaluate_detectors.py

# ConflictBank temporal conflict evaluation
python evaluate_conflictbank.py

# LLM comparison on ConflictBank
python llm_vs_ours_v2.py

# Counterfactual analysis (supersession vs overwrite)
MEM_DB=./synthetic_memory.db python counterfactual.py

# Downstream experiment (expanded, n=300)
python downstream_expanded.py

# Cross-entity downstream (11 genuine contradictions)
python cross_entity_downstream.py
```

Environment: Python 3.13, SQLite 3.x, GLM-5.3-flash via local API (temperature=0), Intel CPU, 32GB RAM.

References

[Chhikara et al.(2025)] Chhikara, P. et al. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv:2504.19413*, 2025.

[Rasmussen et al.(2025)] Rasmussen, R. et al. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv:2501.13956*, 2025.

[Packer et al.(2023)] Packer, C. et al. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*, 2023.

[TOKI(2026)] Anonymous. TOKI: A Bitemporal Operator Algebra for Contradiction Resolution in LLM-Agent Persistent Memory. *arXiv:2606.06240*, 2026.

[MemStrata(2026)] Yadav, N. Temporal Validity in Retrieval Memory: Eliminating Stale-Fact Errors for AI Agents. *arXiv:2606.26511*, 2026.

[Sleeper(2026)] Abdelnabi, S. et al. Hidden in Memory: Sleeper Memory Poisoning in LLM Agents. *arXiv:2605.15338*, 2026.

[AgentPoison(2024)] Chen, S. et al. AgentPoison: Backdoor Attacks on Agent Memory. *NeurIPS*, 2024.

[Su et al.(2024)] Su, Z. et al. ConflictBank: A Benchmark for Evaluating the Influence of Knowledge Conflicts in LLM. *arXiv:2408.12076*, 2024.

[Thorne et al.(2018)] Thorne, J. et al. FEVER: a large-scale dataset for Fact Extraction and VERification. *arXiv:1803.05355*, 2018.

[HaddadPajouh et al.(2026)] HaddadPajouh, H. et al. The Immutable Past: Formalizing State Mutability and Conflict Resolution in Mutable RAG. *arXiv:2609.16073*, 2026.

[Volkov et al.(2026)] Volkov, S. et al. StateFuse: Deterministic Conflict-Preserving Memory for Multi-Agent Systems. *arXiv:2607.05844*, 2026.

[Wang et al.(2026)] Wang, Y. et al. PACMI: Provenance-Aware Cascading Memory Invalidation for Long-Term LLM Agents. *arXiv:2610.05732*, 2026.

[Shen et al.(2026)] Shen, Y. T. et al. Revoked but Still Authoritative: An Empirical Study of Revocation Enforcement in Agent Memory. *arXiv:2609.08258*, 2026.

[Gu et al.(2026)] Gu, H. et al. Whose Memory Is It? Scope-Aware Commit Rules for Long-Term LLM Memory. *arXiv:2610.09008*, 2026.